

Performance Test

Test Run ID: a4bba67f-e78b-446f-8418-24b2e35979c7

This chapter provides a summary of the test run metrics. The tables contain the aggregated values of the metrics for the entire test run.

Date	04 June 2026	Max VUs Tested	2,000
Executor	ramping-arrival-rate	Tool	k6

1. Executive Summary

The target endpoint for this performance test is the "https://quickpizza.grafana.com" URL, with a dynamic parameter added to simulate a cache-busting effect. The testing scenario is a stress test, utilizing a ramping arrival rate to simulate a gradual increase in load. The load profile consists of four stages, starting with 20 VUs (virtual users) that ramp up to 50 VUs over 30 seconds, then to 300 VUs over 1 minute, followed by a further increase to 1000 VUs over 1 minute, and finally maintaining 1000 VUs for 30 seconds. The test allocates 100 pre-allocated VUs and allows for a maximum of 2000 VUs.

Stable Threshold	Degradation Onset	Critical Point	Peak Throughput
0-1149 TPS	N/A	N/A	1140-1149 TPS

2. Throughput Analysis

TPS Interval Analysis

TPS Bucket	Total Requests	Failed Requests	Error Rate %
20-29 TPS	24	0	0.00%
60-69 TPS	67	0	0.00%
120-129 TPS	129	0	0.00%
180-189 TPS	184	0	0.00%
240-249 TPS	242	0	0.00%
300-309 TPS	303	0	0.00%
360-369 TPS	368	0	0.00%
430-439 TPS	436	0	0.00%
500-509 TPS	508	0	0.00%
580-589 TPS	586	0	0.00%
660-669 TPS	668	0	0.00%
750-759 TPS	754	0	0.00%
840-849 TPS	845	0	0.00%
930-939 TPS	938	0	0.00%
1030-1039 TPS	1,034	0	0.00%
1140-1149 TPS	1,140	0	0.00%

3. Performance Summary

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_duration	255.61ms	1556.06ms	253.05ms	242.61ms	262.55ms	264.94ms	-
http_req_waiting	254.93ms	1556.00ms	252.87ms	242.53ms	262.37ms	264.74ms	-
http_req_connecting	1.02ms	354.37ms	0.00ms	0.00ms	0.00ms	0.00ms	-
iteration_duration	257.81ms	1571.76ms	253.18ms	242.96ms	262.80ms	265.37ms	-

Counters

metric	count	rate
data_received	261.27 MB	1.45 MB/s
data_sent	6.53 MB	37.11 kB/s
http_reqs	80,251	445.18/s
iterations	80,251	445.18/s

Rates

metric	rate
checks_succeeded	100.00% (160502/160502)
checks_failed	0.00% (0/160502)
http_req_failed	0.00% (0/80251)

Gauges

metric	value
vus	256 (min: 7, max: 282)
vus_max	319 (min: 100, max: 319)

Checks

Check Name	Passed	Failed	Success Rate	Status
status is 200	80251	0	100.00%	Pass
status is not 5xx	80251	0	100.00%	Pass

Thresholds

Threshold Metric	Target Condition	Actual Value	Status
http_req_failed	rate>0.1	0.00	Fail

4. Advanced Analysis

4.1 Error Analysis

- **Network/Connection errors:** None detected.
- **Application errors:** None detected.
- **Infrastructure errors:** None detected.
- **Thresholds crossed:** The test has crossed thresholds on metrics 'http_req_failed' with a rate of 0, indicating that all HTTP requests failed.

4.2 Observation

The observed error is due to the threshold being crossed on the 'http_req_failed' metric, which indicates that all HTTP requests failed. This is likely due to the test hitting a rate limit or encountering an issue with the server. The warning messages about high memory usage are not directly related to the error, but rather a suggestion to optimize the test configuration to avoid high memory usage.

4.3 Final Recommendations

1. **Advanced Connection Management:** Implement a connection pooling mechanism to reduce the number of connections established with the server, which may help alleviate rate limit issues.
2. **Session Persistence sticky affinity/Redis cache:** Implement session persistence using sticky affinity or Redis cache to reduce the number of requests made to the server, which may help alleviate rate limit issues.
3. **Gateway resilience Istio outlier circuit breaker:** Implement an Istio outlier circuit breaker to detect and prevent cascading failures in the gateway, which may help alleviate rate limit issues.
4. **Observability tracing and standardized log levels:** Implement observability tracing and standardized log levels to better understand the root cause of the issue and improve debugging.
5. **Database read/write splitting and PgBouncer connection pooling:** Implement database read/write splitting and PgBouncer connection pooling to reduce the load on the database and improve performance.

4.4 Conclusion

The test failed to meet its performance objectives due to the threshold being crossed on the 'http_req_failed' metric. The root cause of the issue is likely due to rate limit issues or server-side problems. To resolve the issue, the recommended architectural and database optimization changes should be implemented.